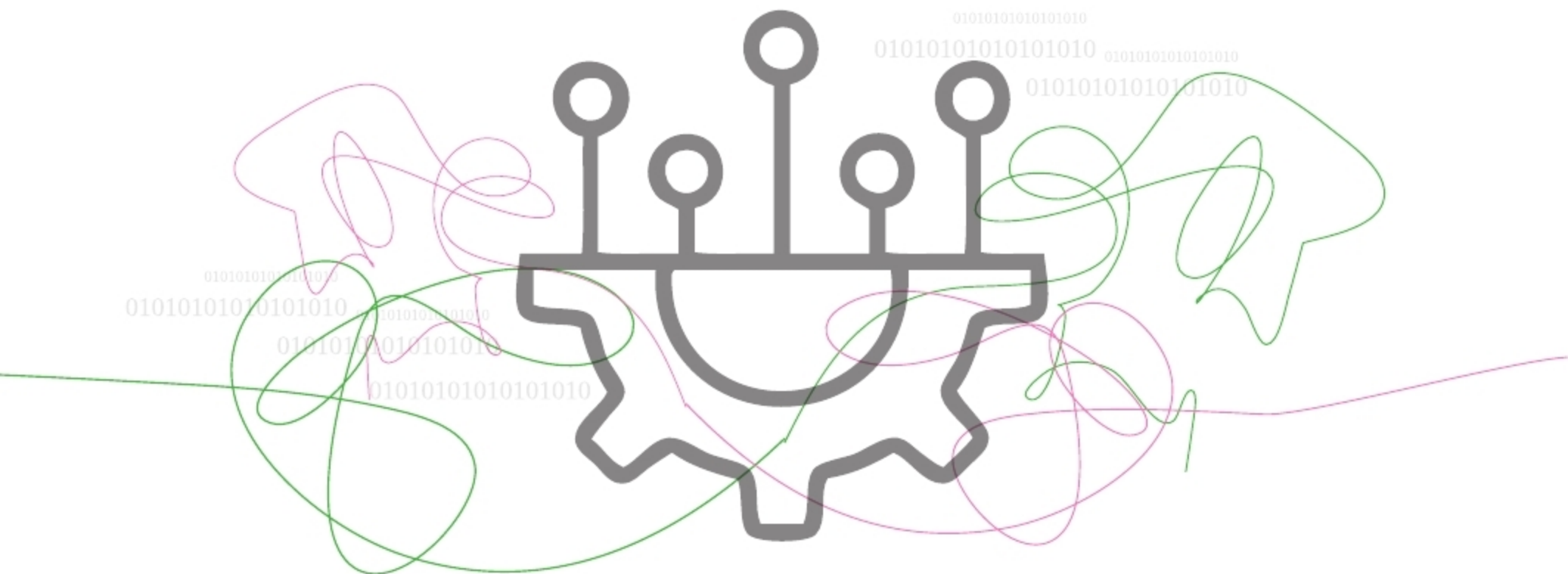


Solutions to Priority Inversion in a Multi-threaded Programming Environment

Priority inversion occurs in priority-based scheduling. This results in slower performance and unexpected results in general-purpose operating systems. This article demonstrates the use of Pthread locking mechanisms to simulate the priority inversion problem and how mutex attributes, called protocols, can be used to avoid priority inversion problems.



Multi-threading has become a widely used programming technique in the design of modern software. This results in the increased responsiveness of the application and the effective use of multi-processor systems. Threads share several resources such as address space, code section and data section, which enables them to achieve faster context switches and easier communication. However, this sharing of resources leads to synchronisation and scheduling problems. One such problem is priority inversion, which occurs in priority-based scheduling. In general-purpose OSs, this results in slower performance and unexpected results.

Many real-time operating systems use pre-emption to ensure rapid response times. This allows a higher priority task to pre-empt a lower priority task that is running. Once the higher priority task is complete, the lower priority task can resume its execution from the point where it was pre-empted.

This pre-emption guarantees worst case performance in safety critical situations. The need to share resources between tasks running in a pre-emptive multi-tasking environment can lead to problems such as deadlock and priority inversion.

In 1997, one such problem arose in the Mars Pathfinder. The rover was collecting meteorological data on Mars. Because of priority inversion, the spacecraft started resetting itself, which resulted in data loss and delays in data collection. The problem was fixed in a few days by patching the onboard software.

Mutex locks provide synchronisation mechanisms between threads. They prevent data inconsistencies that arise due to race conditions. It is a lock that is set before using a shared resource and then released once the resource is used. Whenever the lock is set, no other thread can access the locked region of the code. This ensures synchronised access of shared resources. One such mutex is Pthread mutex, and this locking mechanism supports the priority inheritance protocol.